Phishing URL Detection Using Python

Project Report
Prepared using Python and Machine Learning
Date: 21 October 2026

1. Abstract

Phishing attacks trick users into revealing sensitive information such as passwords, banking credentials, and personal data through fake websites. This project builds a Python-based phishing URL detection system that analyzes URL features and classifies a link as either **legitimate** or **phishing**. The system uses machine learning, making it suitable for browser extensions, email security tools, and web applications.

2. Objectives

- Detect phishing URLs automatically using Python. - Extract important lexical, host-based, and content-related URL features. - Train and evaluate a machine-learning classification model. - Provide a simple prediction interface for new URLs. - Measure accuracy, precision, recall, F1-score, and confusion matrix.

3. Problem Statement

Users often cannot visually distinguish a malicious URL from a genuine one. Attackers use look-alike domains, shortened links, excessive subdomains, HTTP instead of HTTPS, suspicious symbols, and misleading keywords. A manual inspection approach is slow and unreliable; therefore, an automated intelligent detection system is required.

4. Existing System and Limitations

Traditional systems rely mainly on blacklists of known malicious domains. They are easy to deploy but cannot detect newly created or slightly modified phishing URLs. Rule-based systems improve detection but require frequent manual updates and may produce false positives.

5. Proposed System

The proposed system collects URLs, extracts meaningful features, and passes them to a supervised machine-learning model. The model learns patterns commonly found in phishing URLs and predicts the class of unseen URLs. Python libraries such as **scikit-learn**, **pandas**, **NumPy**, and **requests** are used for data processing, model training, and evaluation.

6. System Architecture

Input URL → Preprocessing → Feature Extraction → Machine-Learning Model → Classification → Result (Legitimate / Phishing)

7. URL Features Used

Feature	Description
URL length	Phishing URLs are often unusually long.
Hostname length	Long or complex hostnames may indicate deception.
Number of dots	Too many dots can indicate misleading subdomains.
Use of IP address	Direct IP-based URLs are often suspicious.
HTTPS presence	Missing HTTPS can be a risk indicator.
@ symbol	An @ symbol can hide the real destination domain.
Hyphen count	Excess hyphens are common in fake domains.
Suspicious words	Words such as login, verify, secure, and account may appear in phishing links.
Shortening service	Shortened URLs can conceal the actual destination.
Digits in URL	High digit usage may indicate automatically generated malicious links.

8. Technologies Used

- **Programming language:** Python 3 - **Data handling:** pandas, NumPy - **Machine learning:** scikit-learn - **Model:** Random Forest Classifier / Logistic Regression - **Evaluation:** accuracy, precision, recall, F1-score, confusion matrix - **Optional interface:** Flask or Streamlit

9. Sample Python Implementation

The following example demonstrates feature extraction and model training using scikit-learn.

```
import pandas as pd from sklearn.model_selection import train_test_split from sklearn.ensemble import RandomForestClassifier from sklearn.metrics import accuracy_score, classification_report # dataset.csv must contain URL features and a label column # label: 1 = phishing, 0 = legitimate data = pd.read_csv("dataset.csv") X = data.drop(columns=["label"]) y = data["label"] X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42, stratify=y ) model = RandomForestClassifier(n_estimators=200, random_state=42) model.fit(X_train, y_train) y_pred = model.predict(X_test) print("Accuracy:", accuracy_score(y_test, y_pred)) print(classification_report(y_test, y_pred)) def extract_features(url): return { "url_length": len(url), "num_dots": url.count("."), "has_at_symbol": int("@" in url), "uses_https": int(url.startswith("https")), "num_hyphens": url.count("-"), "has_ip": int(any(part.isdigit() for part in url.split("//")[-1].split("/")[0].split("."))) } new_url = "http://secure-login.example.com/verify" features = pd.DataFrame([extract_features(new_url)]) prediction = model.predict(features)[0] print("Phishing" if prediction == 1 else "Legitimate")
```

10. Model Evaluation

- **Accuracy:** Overall percentage of correctly classified URLs. - **Precision:** Of all URLs predicted as phishing, how many were actually phishing. - **Recall:** Of all actual phishing URLs, how many the model detected. - **F1-score:** Balance between precision and recall. - **Confusion matrix:** Shows true positives, true negatives, false positives, and false negatives.

11. Advantages

- Detects suspicious URLs before a user visits them. - Can identify newly generated phishing patterns using machine learning. - Fast, automated, and scalable. - Can be integrated into browsers, email filters, and mobile applications. - Reduces dependence on manually maintained blacklists.

12. Limitations

- Attackers continuously change URL patterns. - A URL may look safe but host malicious content after redirection. - False positives can occur for unusual but legitimate URLs. - Model quality depends heavily on a balanced and updated dataset.

13. Future Enhancements

- Add real-time domain-age, WHOIS, DNS, and SSL certificate checks. - Use deep-learning models for sequence-based URL analysis. - Develop a browser extension for live warning messages. - Add a Flask or Streamlit web interface. - Continuously retrain the model using newly reported phishing URLs.

14. Conclusion

This project demonstrates how Python and machine learning can be used to detect phishing URLs effectively. By extracting URL-based features and training a classifier, the system can help users avoid malicious websites and reduce the risk of credential theft. With continuous dataset updates and additional security signals, the solution can become a practical component of modern cybersecurity systems.